Genomgång - NoSQL med MongoDB

Olika typer av NoSQL (Not Only SQL)

- Dokument orienterade databaser (MongoDB)
- Nyckel-värde databaser (Redis)
- Kolumn orienterade databaser (Cassandra)
- Graf orienterade databaser (Neo4j)

Vi skall fokusera på dokument orienterade databaser med specifikt MongoDB

Skillnaden mellan MySQL (RDBMS) och MongoDB (NoSQL) är

	RDBMS	**NoSQL**
Databasmodell	Tabell baserad databas modell	Varierade databasmodeller såsom Dokument, nyckel-värde, kolumn o graf.
Skalbarhet	Begränsad horisontell skalbarhet. (det svårt att hantera stora belastningar	Obegränsad horisontell och vertikal skalbarhet. Ideellt för att hantera större mängder data.
Schema	Har en strikt fördefinierad struktur som måste efterföljas	Mer flexibel struktur, som tillåter att nya fält kan läggas till i efterhand.
Säkra Transaktion	Stöd för transaktioner (ACID-egenskaper: Atomicity, Consistency, Isolation, Durability), garanterar dataintegritet. Om ett fel skulle uppstå under transaktionen, då kör databasen en rollback automatiskt	Varierande stöd beroende på typen av NoSQL databas. Men i de flesta fallen prioriteras tillgänglighet och snabbhet över säkra transaktioner.
Sammanfattning	Passar bra för applikationer med komplexa relationer mellan data, samt behov av säkra transaktioner. Exempelvis e-handel och affärssystem	Passar bra för stora datamängder, molnbaserade applikationer, realtidsanalyser, IoT (Internet of Things) och där flexibilitet och skalbarhet är avgörande.

Skapa kontot i MongoDB Atlas

https://www.mongodb.com/cloud/atlas/register

Installera MongoDB Compass

https://www.mongodb.com/try/download/compass

Intro CRUD med MongoDB via Terminalen

INTRO - Jobba med DBs och Collections

//Connect to DB server
mongosh "mongodb+srv://**<user>**:**<password>**@cluster0.wffly.mongodb.net/"

```
show dbs                          // Display all DBs
use <db_name>                     // Work with specific DB
show collections                  // Display all collections for a DB
db.createCollection("students")   // Skapar en "students" collection
```

classes

_id	**name**
xxx	Klass A
xxx	Klass B
xxx	Klass C
xxx	Klass D

students

_id	**fname**	**age**	**class_id**
xxx	Beatrice	24	xxx (id for Klass A)
xxx	Mohamad	26	xxx (id for Klass C)
xxx	Johanna	29	xxx (id for Klass B)
xxx	Mohamad	35	xxx (id for Klass B)
xxx	Ivan	24	xxx (id for Klass A)

INSERT

```
db.classes.insertOne({"name": "Klass A"})
db.classes.insertOne({"name": "Klass B"})
db.classes.insertOne({"name": "Klass C"})
db.classes.insertOne({"name": "Klass D"})
```

```
db.students.insertMany([
        {"fname": "Beatrice", "age": 24, "class_id": ObjectId("xxx")},
        {"fname": "Mohamad", "age": 26, "class_id": ObjectId("xxx")},
        {"fname": "Johanna", "age": 29, "class_id": ObjectId("xxx")},
        {"fname": "Mohamad", "age": 35, "class_id": ObjectId("xxx")},
        {"fname": "Ivan", "age": 24, "class_id": ObjectId("xxx")}
])
```

Operators

The following operators can be used in the WHERE clause:

Operator	Description
`$eq`	Equal
`$ne`	Not equal.
`$gt`	Greater than
`$lt`	Less than
`$gte`	Greater than or equal
`$lte`	Less than or equal
`$in`:	Value is matched within an array
`$regex`	Allows the use of regular expressions when evaluating field values
$and	Returns documents where both queries match
$or	Returns documents where either queries match
$nor	Returns documents where both queries fail to match
$not	Returns documents where the query does not match

SELECT/FIND

`db.collection.find(query, projection, options)`

```
// Find all
db.students.find()

// Find all students with fname "Mohamad"
db.students.find({"fname": "Mohamad"})

// Find all, display "fname" and "age" fields
db.students.find({}, {_id: 0, fname: 1, age: 1})

// Find all, display all fields except for "fname"
db.students.find({}, {fname: 0})
```

```
// Find one, first match
db.students.findOne()

// Find one (first match) with fname "Mohamad"
db.students.findOne({"fname": "Mohamad"})
```

SELECT/FIND WITH COMPARISON/EVALUATION OPERATORS

```
// Find all students with fname is NOT "Beatrice"
db.students.find({"fname": {"$ne": "Beatrice"}})

// Find all students with age greater than 24
db.students.find({"age": {"$gt":24}})

// Find all students with age greater than OR equal to 24
db.students.find({"age": {"$gte":24}})

// Find all students with age lesser than 26
db.students.find({"age": {"$lt":26}})

// Find all students with age lesser than OR equal to 26
db.students.find({"age": {"$lte":26}})

// Find all students with age 26, 29
db.students.find({"age": {"$in": [26, 29]}})

// Find all students with an “an” part in their fname
db.students.find({"fname": {"$regex": "an"}})
```

SELECT/FIND WITH LOGICAL OPERATORS

```
// Find all students with fname "Johanna" OR age 24
db.students.find({"$or": [{"fname": "Johanna"}, {"age": 24}]})

// Find all students with fname "Mohamad" AND age 26
db.students.find({"$and": [{"fname": "Mohamad"}, {"age": 26}]})
// Same as following
db.students.find({"fname": "Mohamad", "age": 26})
// But $and can be used in a wider range of situations
db.students.find({"$or": [{"$and": [{"fname": "Mohamad"}, {"age": 26}] }, {"age": 24}]})
// If condition is rewritten in JS
// const contition = (fname = “Mohamad” && age = 26) || age =  24

// Find all students with NOT fname "Johanna" NOR age 24 - Exact opposite of OR
db.students.find({"$nor": [{"fname": "Johanna"}, {"age": 24}]})

// Find all students that are NOT greater than 25 of age
db.students.find({"age" : {"$not": {"$gt": 25}}})

// Difference between $not and $ne:
https://www.mongodb.com/docs/manual/reference/operator/query/not/
```

SELECT/FIND WITH READ OPERATORS

```
db.students.find().sort({"fname": -1})  // Sort by fname descending order
db.students.find().sort({"fname": 1})   // Sort by fname ascending order
db.students.find().limit(3)             // Limit to first 3 students
db.students.find().sort({"fname": -1}).limit(3) // May chain multiple methods together
```

UPDATE

`db.collection.update(query, update, options)`

```
 // Update one -> Update one student with fname “Johanna” to age 30
db.students.updateOne( { fname: "Johanna" }, { $set: { age: 30 } } )

// Update many -> all students with age 24 is updated to age 25
db.students.updateMany( { age: 24 }, { $set: { age: 25 } } )

// Update one with …, if not found insert it (By setting upsert=true)
db.students.updateOne(
  { fname: "Maria" },
  {$set: { fname: "Maria", age: 44}},
  { upsert: true }
)
```

DELETE

`db.collection.deleteOne()`

```
db.students.deleteOne({ fname: "Maria" })
db.students.deleteMany({ age: 25 })
```

AGGREGATE

```
// Find first 2 students
db.students.aggregate([{$limit: 2}])

 // students sorted after fname in ascending order, and limited to first 3 students
db.students.aggregate([{$sort: {"fname": 1}}, {$limit: 3}])

// Finds students with a specific fname
db.students.aggregate({$match: {"fname": "Mohamad"}})

// Find all unique student names, that has students
db.students.aggregate([{$group: {_id:"$fname"}}])

// Find max age of students
db.students.aggregate([{ $group: { _id: null, maxAge: { $max: "$age" } } }]);

// Count all students older than 24 years
db.students.aggregate([
  {$match: { "age": {"$gt":24}}},
  {$count: "Students"}
])
```

Advanced Aggregation

```
// Join "students" with "classes", find students with specific fname,
// display only fields given in $project
db.students.aggregate([
        {$lookup: {
            from: "classes",
            localField: "class_id",
            foreignField: "_id",
            as: "class_details",
        }},

        {$unwind: "$class_details" },
        {$match: {"fname": "Mohamad"}},

        {$project: {
            "fname": 1,
            "age": 1,
            "class_name": "$class_details.name"
         }}
])

// Find all classes that has students, with student count for each class
db.students.aggregate([
        {$lookup: {
            from: "classes",
            localField: "class_id",
            foreignField: "_id",
            as: "class_details",
        }},
        { $unwind: "$class_details" },

        {$group: {
                _id: "$class_details._id",
                student_count: { $sum: 1 },
                class_name: { $first: "$class_details.name" }
        }},
])
```

Läsanvisningar:
Operationer för MongoDB, från W3Schools:

- MongoDB intro
- Get started
- Create DB
- Collection
- Insert
- Find
- Update
- Delete
- Query operators
- Update operators
- Aggregations
- Validation

Från MongoDB Documentation

- https://www.mongodb.com/docs/manual/reference/operator/query/’
- https://www.mongodb.com/docs/manual/tutorial/insert-documents/
- https://www.mongodb.com/docs/manual/tutorial/update-documents/
- https://www.mongodb.com/docs/manual/reference/method/js-collection/
- https://www.mongodb.com/docs/manual/aggregation/